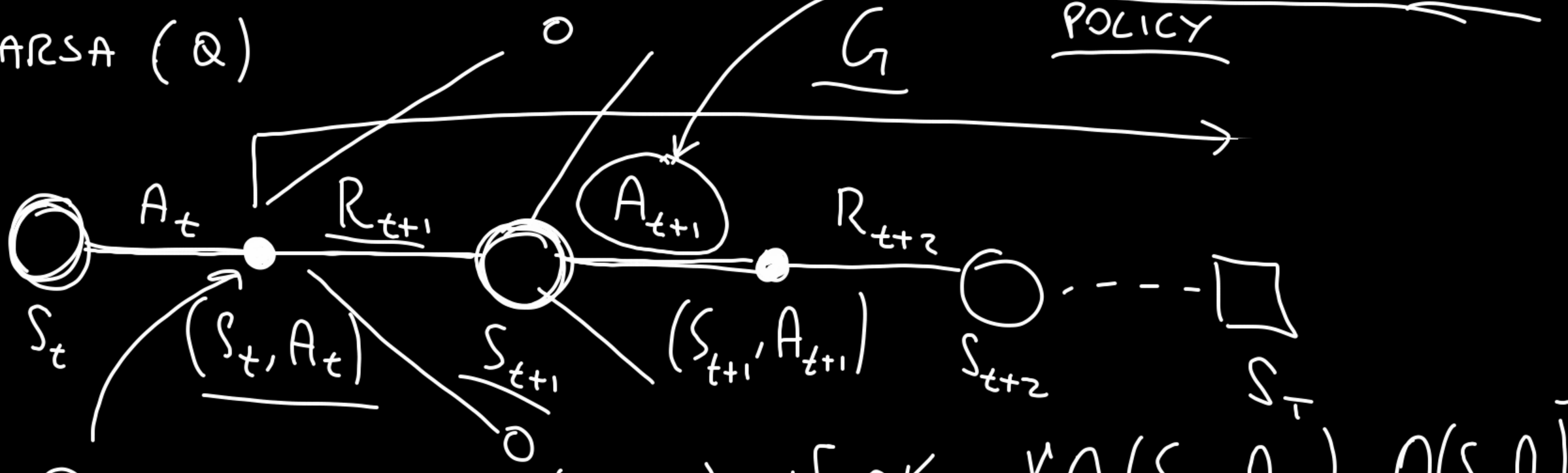
12/11/26 - Q-LEARNING / DEEP Q-LEARNING

TEMPORAL DIFFERENCE (TD)

SARSA (Q)

SELECTED WITH ε-GREEDY POLICY

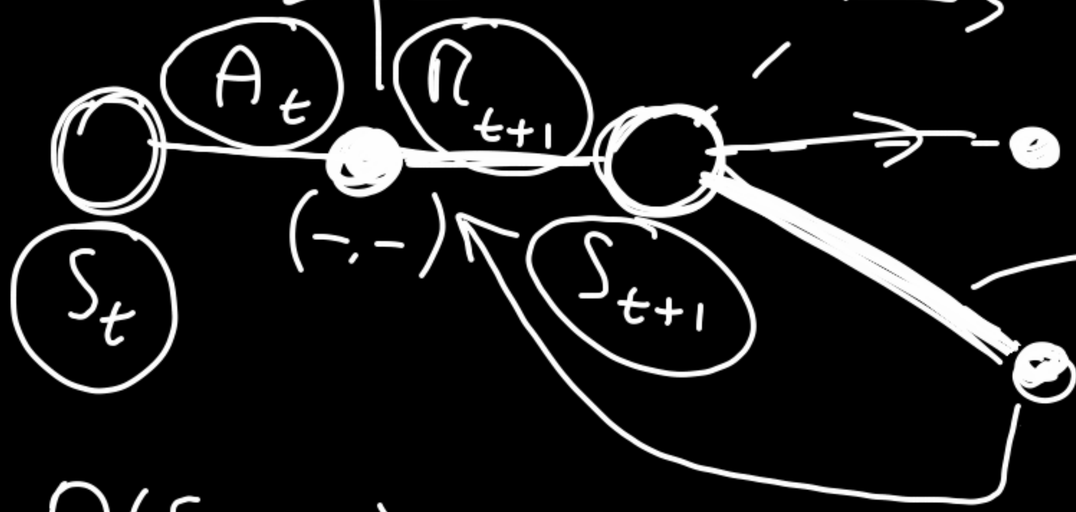


$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[\underbrace{R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})}_{\text{TARGET}} - Q(S_t, A_t)\right]$$

$$E\{\text{TARGET} \mid S_t = s, A_t = a\} = \sum_{s', r} p(s', r \mid s, a)\left[r + \gamma \sum_{a'} \pi(a \mid s')\, q_\pi(s', a')\right]$$

Q-LEARNING

ε-GREEDY (Q) / π_b



$$A^* = \underset{a}{\operatorname{argmax}}\, Q(S_{t+1}, a)$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[\underbrace{R_{t+1} + \gamma\, Q(S_{t+1}, A^*)}_{\text{TARGET}} - Q(S_t, A_t)\Big]$$

Q-LEARNING

$Q \longrightarrow q^*$

$$\text{TARGET} = \Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a)\Big]$$

DOES NOT DEPEND ON POLICY USED TO EXPLORE

$E\{\text{TARGET}\}$ = RIGHT-HAND SIDE OF BELLMAN OPT. EQ.

OVERESTIMATION OF THE VALUE FUNCTION Q

DOUBLE Q-LEARNING TWO Q ESTIMATIONS Q_1, Q_2

$$T = R_{t+1} + \gamma \max_{a} Q(S_{t+1}, a) =$$

$$= R_{t+1} + \gamma\, Q\left(S_{t+1}, \operatorname{argmax}_{a}\{Q(S_{t+1}, a)\}\right)$$

(ACTOR/CRITIC METHODS.)

DOUBLE Q-LEARNING

$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$ — UPTATE RANDOMLY ONE OF THE TWO.

- $Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha\left[R_{t+1} + \gamma Q_1\left(S_{t+1}, \underbrace{\underset{a}{\operatorname{argmax}}\left\{Q_2(S_{t+1}, a)\right\}}_{\text{ACTOR}}\right) - Q_1(S_t, A_t)\right]$ CRITIC

- $Q_2(S_t, A_t) \leftarrow Q_2(S_t, A_t) + \alpha\left[R_{t+1} + \gamma Q_2\left(S_{t+1}, \underset{a}{\operatorname{argmax}}\left\{Q_1(S_{t+1}, a)\right\}\right) - Q_2(S_t, A_t)\right]$

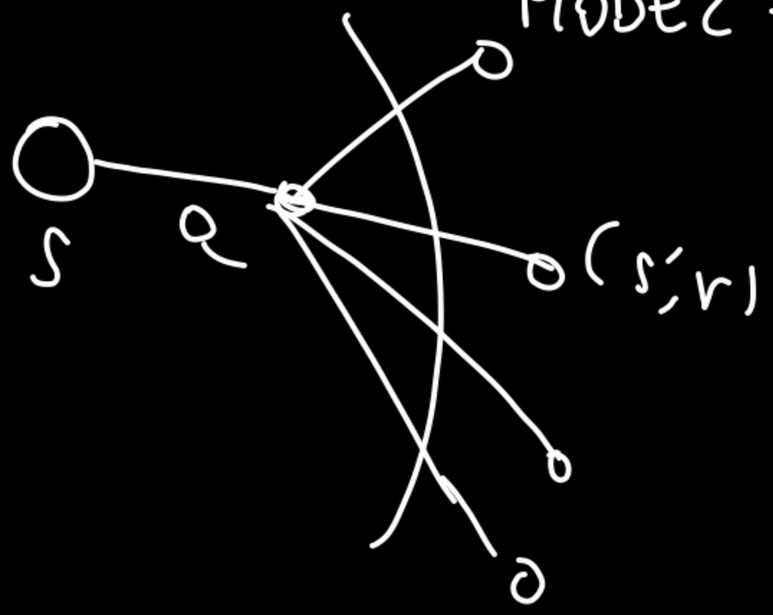
ε-greedy POLICY $\left(\frac{Q_1 + Q_2}{2}\right)$

FINAL REMARKS

- DYNAMIC PROGRAMMING (P, MODEL OF ENV. IS KNOWN)

PLANNING: USE OF A MODEL TO IMPROVE THE POLICY (OR TO FIND π^*). EXPECTED UPDATES (DISTRIBUTION

MODEL — PROB. DISTR. OF s', r).

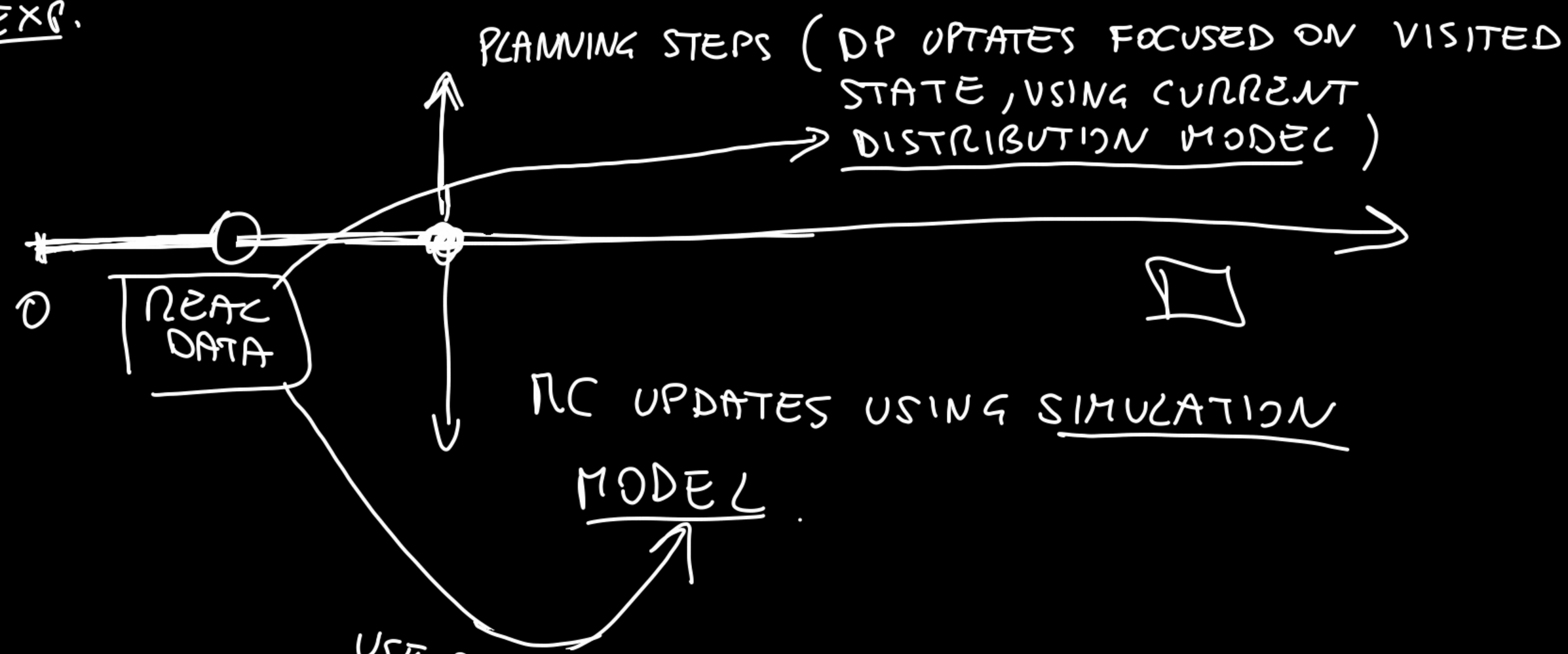


LEARNING : WE IMPROVE THE POLICY BY USING DATA THAT COMES FROM THE INTERACTION WITH ENVIRONMENT.

- INTERACTION WITH REAL ENVIRONMENT (REAL DATA)
- SIMULATED ENVIRONMENT.

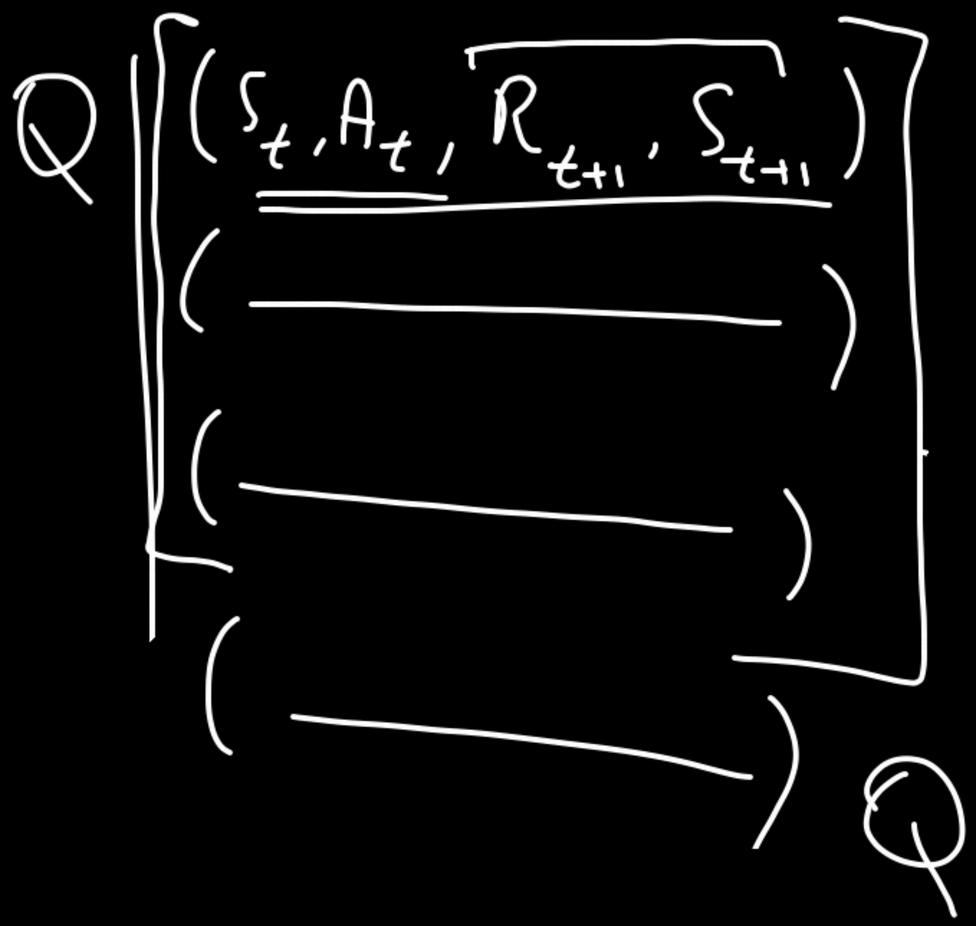
- REAL DATA IS USED TO COMPUTE EMPIRICAL, ITERATIVE AVERAGES OF THE RETURNS (TO ESTIMATE VALUE FUNCTIONS).
- REAL DATA CAN BE USED TO IDENTIFY MODELS OF THE ENVIRONMENT, WHICH CAN BE USED FOR PLANNING.

EXP.


PLANNING STEPS (DP UPTATES FOCUSED ON VISITED STATE, USING CURRENT DISTRIBUTION MODEL)
0
REAL DATA
MC UPDATES USING SIMULATION MODEL
USE REAL DATA TO IMPROVE THE MODELS (IDENTIFICATION)

PUT OBSERVED TRANSITIONS IN A "REPLAY BUFFER" D.

TARGET CALCULATION



DEEP RL

TABULAR METHODS (FINITE MDP  $\rightarrow$ V AND Q CAN BE REPRESENTED AS TABLES). $|S| = 10^{100}$)

NOT PRACTICAL FOR MOST REAL-LIFE APPLICATIONS:

- $|S|, |R|, |A|$ TOO LARGE.
- PROBLEMS WITH CONTINUOUS (NOT DISCRETE) STATES/ACTIONS.

$\Downarrow$

CANNOT BUILD TABLES TO REPRESENT V AND Q $\rightarrow$ WE USE PARAMETRIZED APPROXIMATIONS OF THE VALUE FUNCTIONS

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 10.1 \\ 12.1 \end{pmatrix} \longrightarrow v_\pi \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} ?$$

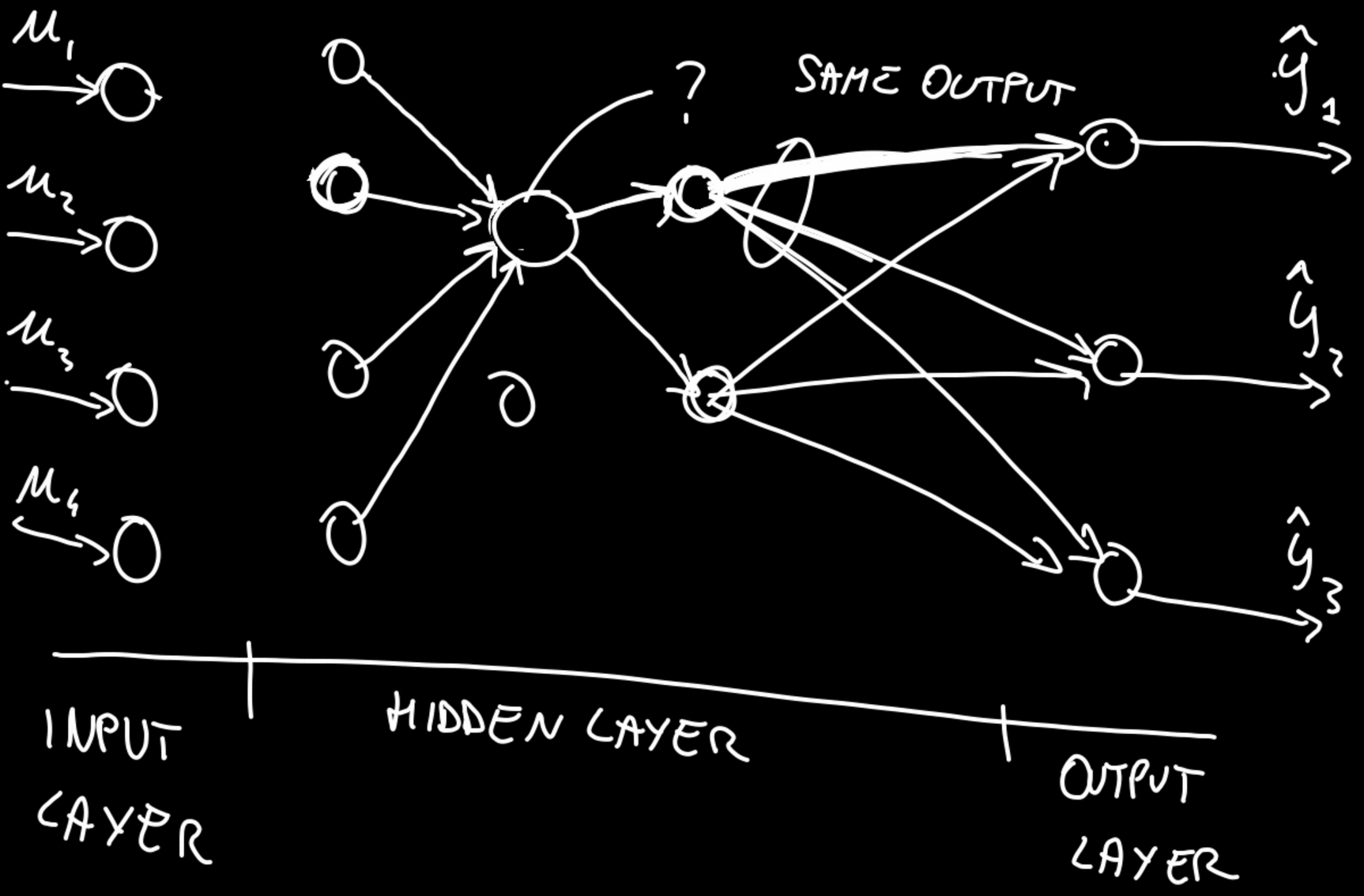
$v_\pi(s, w_v)$

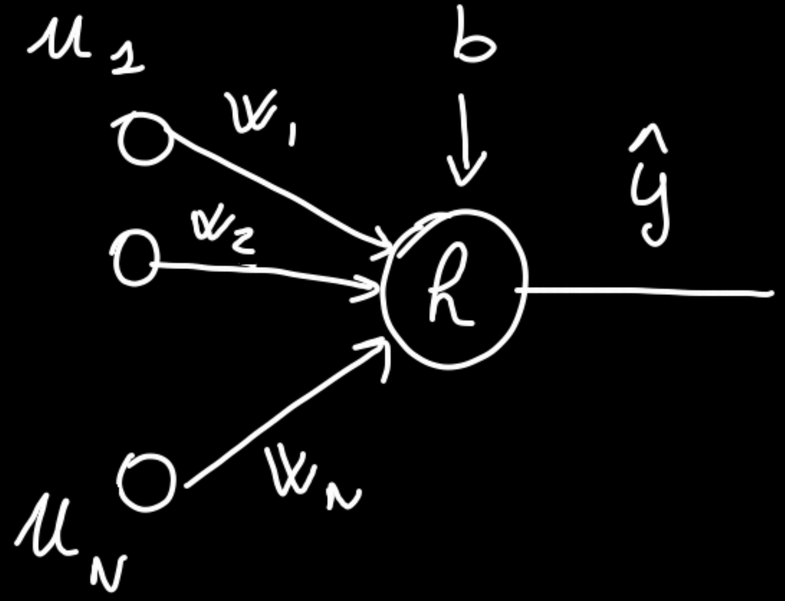
$\longrightarrow$ PARAMETERS OF THE FUNCTIONAL APPROXIMATION

$q_\pi(s, a, w_q)$

NN TO APPROXIMATE THE VALUE FUNCTIONS:

FEED FORWARD NEURAL NETWORK:





LINEAR COMBINATION OF INPUTS FROM PREVIOUS LAYER

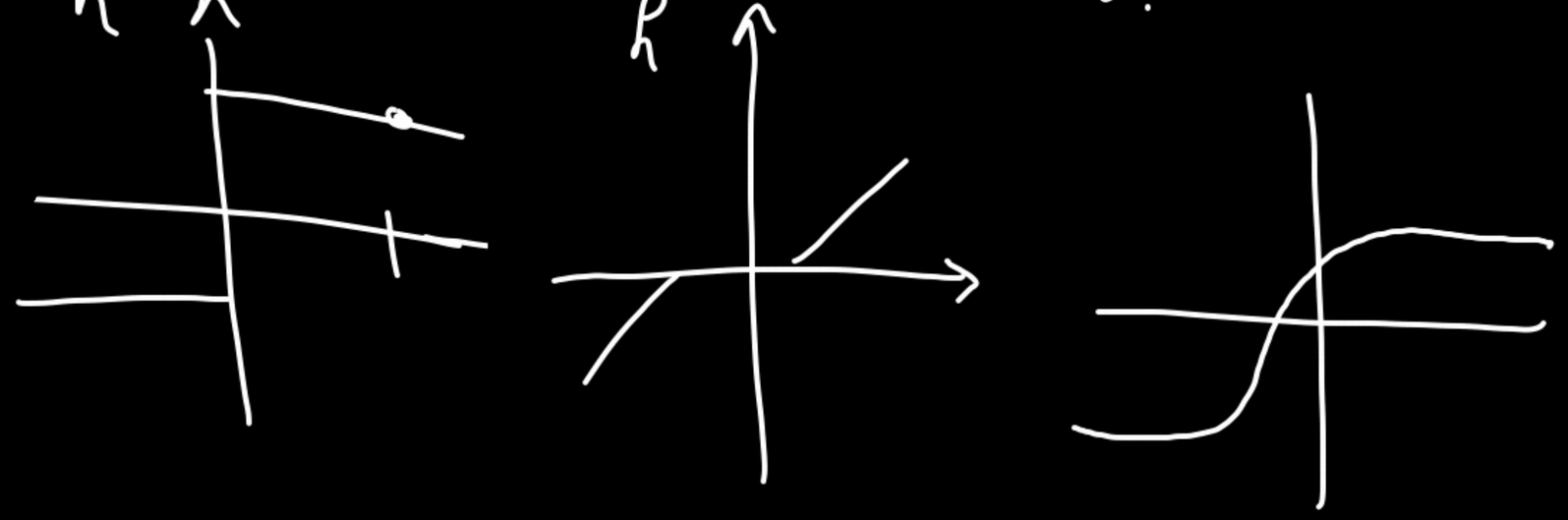
$$y = h\left(\sum_{i=1}^{N} w_i u_i\right) + b$$

ACTIVATION FUNCTION

CONNECTION WEIGHTS.

BIAS

h TYPICALLY NONLINEAR



NN:

- HOW MANY HIDDEN LAYER AND HOW DENSE.
- WEIGHTS (W)
- ACTIVATION FUNCTIONS.

↳ ANY GIVEN CHOICE DEFINES A PARTICULAR MAP FROM INPUTS TO OUTPUTS (A PARTICUCAR FUNCTION).

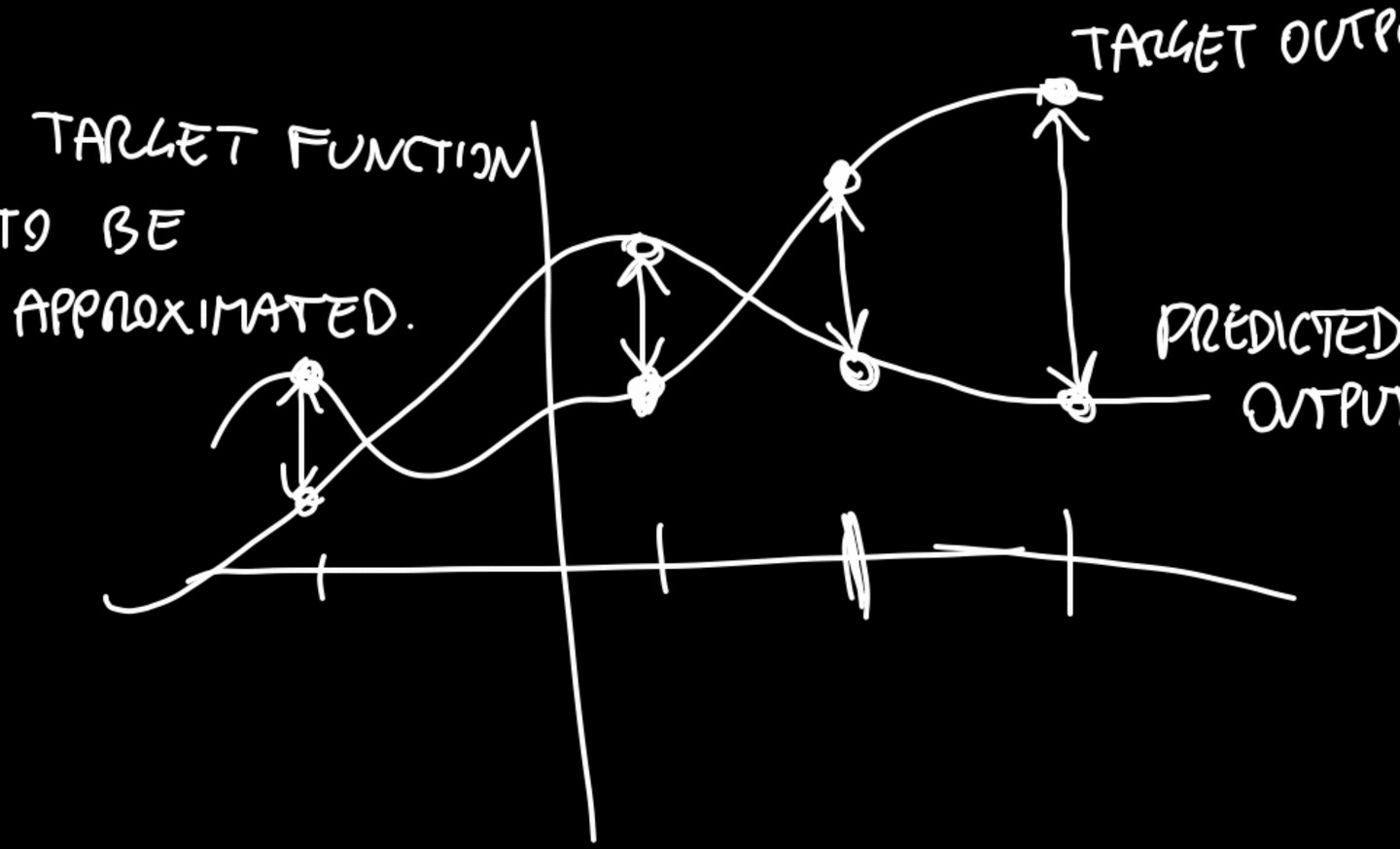
NN ARE UNIVERSAL FUNCTIONAL APPROXIMATORS:

THEY CAN APPROXIMATE ANY FUNCTION WITH A PROPER CHOICE OF THE STRUCTURE / PARAMETERS.

TARGET FUNCTION

TRAINING DATASET

INPUTS	TARGET OUTPUT
u_1^t	y_1^t
u_2^t	y_2^t
...	...



NN TRAINING (SELECTION OF W USING TRAINING DATASED) IS DONE BY SOLVING AN OPTIMIZATION PROBLEM, WHICH AIMS TO MINIMIZE A LOSS FUNCTION.

$$\mathcal{L}(y^t, \hat{y}, W) = \frac{1}{M} \sum_{i=1}^{M} \left(y_i - \hat{y}_i \right)^2$$

- y^t: TARGET
- $\hat{y}$: NN PREDICTION
- M: # OF TRAINING SAMPLES
- $\hat{y}_i =$ FUNCTION OF W

$$W^* = \underset{W}{\operatorname{argmin}}\{d\}$$

NONLINEAR (h ARE NONLINEAR) PROBLEM DIFFICULT TO SOLVE.

IT IS "SOLVED" ITERATIVELY.

ITERATIVE NN TRAINING

$\mathcal{L}(W)$

W^0 (CURRENT WEIGHTS) $\longrightarrow$ W'

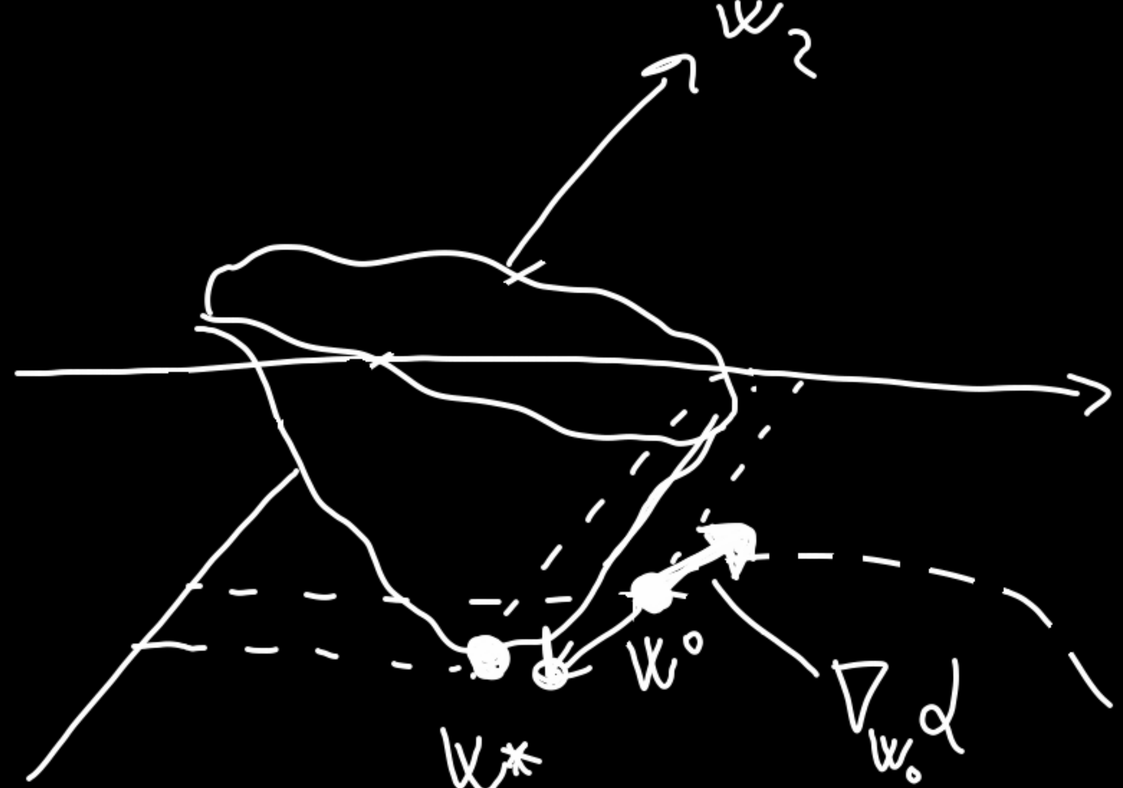
HOW CAN WE FIND NEW WEIGHTS TO IMPROVE (REDUCE) LOSS.

$$\mathcal{L}(W) = \frac{1}{M} \sum_i \left(y_i^T - \hat{y}(W) \right)^2$$

y_i^T: known

GRADIENT - BASED UPDATE RULE.

$\mathcal{L}(w)$



$\mathcal{L} : W \to \mathbb{R}$

$$\nabla_w \mathcal{L} = \frac{\partial \mathcal{L}}{\partial w} = \begin{pmatrix} \frac{\partial \mathcal{L}}{\partial w_1}(w) \\ \frac{\partial \mathcal{L}}{\partial w_2}(w) \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial w_{|w|}}(w) \end{pmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial w}\Big|_{w=w^0}$$

$$w' = w^0 - \alpha \nabla_{w_0} \mathcal{L}$$